

**Частное образовательное учреждение высшего образования
«Международный Институт Дизайна и Сервиса»
(ЧОУВО МИДиС)**

Кафедра математики и информатики

ОТЧЕТ ПО УЧЕБНОЙ ПРАКТИКЕ

Специальность: 09.02.07 Информационные системы и программирование

Квалификация: программист

Форма обучения: очная

Выполнил: Лобанов А. Н.

Группа: П-28

Проверил: Прилепина Е. В.

Челябинск 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	4
ГЛАВА 1 ХАРАКТЕРИСТИКА БАЗЫ ПРАКТИКИ	6
ГЛАВА 2 ОСНОВЫ АЛГОРИТМИЗАЦИИ И ПРОГРАММИРОВАНИЯ	8
2.1 Тестирование программных комплексов	8
2.2 Разработка программной оболочки	9
2.3 Индивидуальное задание. Проектирование и разработка программной оболочки риэлтерского агентства	13
ГЛАВА 3 ОПЕРАЦИОННЫЕ СИСТЕМЫ И СРЕДЫ	17
3.1 Введение в Kotlin и Android Studio	17
3.2 Изучение жизненного цикла Android	17
3.3 Верстка на языке Kotlin	18
ГЛАВА 4 БАЗЫ ДАННЫХ	23
4.1 Инфологическое проектирование баз данных	23
4.2 Логическое проектирование баз данных	27
4.3 Индивидуальное задание. Инфологическое и логическое проектирования баз данных	28
ГЛАВА 5 ЛИНЕЙНОЕ ПРОГРАММИРОВАНИЕ	31
5.1 Экономико-математическая модель	31
5.2 Индивидуальное задание. Построение экономико-математической модели	31
5.3 Графический метод решения ЗЛП	33
5.4 Индивидуальное задание. Решение ЗЛП графическим методом	34
5.5 Симплексный метод решения ЗЛП	36
5.5 Индивидуальное задание. Решение ЗЛП симплексным методом	38
5.6 Решение ЗЛП двойственным методом	39
5.7 Индивидуальное задание. Решение ЗЛП двойственным методом	40
ГЛАВА 6 ПРОЕКТИРОВАНИЕ И РАЗРАБОТКА ИНТЕРФЕЙСОВ	43
6.1 Индивидуальное задание. Верстка сайта «Мой не сам»	43
6.2 Валидация форм сайта «Мой не сам»	44
6.3 Flexbox-верстка сайта «Интерьер»	45
6.4 Слайдер HTML, CSS и JavaScript	47
6.5 Создание псевдотрехмерного пространства в CSS	49
6.6 Индивидуальное задание. Верстка сайта «Языковой портал»	50

ЗАКЛЮЧЕНИЕ	54
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	56
ПРИЛОЖЕНИЕ А	58

ВВЕДЕНИЕ

Учебная практика направлена на формирование у обучающихся практических профессиональных умений, приобретение первоначального практического опыта, реализуется в рамках профессиональных модулей образовательной программы по основным видам профессиональной деятельности для последующего освоения ими общих и профессиональных компетенций по специальности 09.02.07 Информационные технологии и программирование.

Учебная практика организуется в форме практической подготовки и предусмотрена календарным учебным графиком в течении 5 недель в рамках профессиональных модулей по специальности 09.02.07 Информационные технологии и программирование.

Главной целью учебной практики является комплексное освоение обучающимися всех видов профессиональной деятельности по специальности 09.02.07 Информационные технологии и программирование, формирование общих и профессиональных компетенций, а также приобретение необходимых умений и опыта практической работы по специальности 09.02.07 Информационные технологии и программирование.

Задачи учебной практики:

- обучение студентов трудовым приемам, операциям и способам деятельности характерным для соответствующей профессии и необходимым для последующего освоения ими общих и профессиональных компетенций по избранной профессии;
- формирование общих и профессиональных компетенций;
- закрепление и совершенствование первоначальных практических профессиональных умений студентов;
- Проектирование и разработка программной оболочки риэлтерского агентства;
- Разработка мобильного приложения на операционной системе Android;

- Инфологическое и логическое проектирования;
- Верстка сайтов «Мой не сам» и «Языковой портал»;
- Построение математическое модели;
- Графический метод решения ЗЛП;
- Симплексный метод решения ЗЛП;
- Двойственный метод решения ЗЛП;

Программа учебной практики является частью программы образовательной программы по специальности 09.02.07 Информационные системы и программирование в части освоения основных видов профессиональной деятельности:

- Проектирование и разработка информационных систем;
- Разработка дизайна веб-приложений;
- Проектирование, разработка и оптимизация веб-приложений.

ГЛАВА 1 ХАРАКТЕРИСТИКА БАЗЫ ПРАКТИКИ

Частное Образовательное Учреждение Высшего Образования «Международный Институт Дизайна и Сервиса» основано 16 января 1992 года.

Основным видом деятельности является образовательная деятельность. МИДиС включает в себя общеобразовательную школу «7 ключей», колледж и институт.

Система управления МИДиС включает в себя: ректорат, деканат, бухгалтерию, сельскохозяйственный и образовательный отделы. МИДиС занимается обучением по направлениям:

1. программирования;
2. дизайна (дизайн одежды, веб-дизайн, промышленный дизайн);
3. бизнес-информатики;
4. гостиничного дела;
5. туризма;
6. торгового дела;
7. сервиса.

Кабинеты ЧОУВО «МИДиС» оборудованы современными персональными компьютерами с актуальными компонентами и обширным количеством ПО, например:

1. Unity;
2. ZBrush;
3. Blender;
4. Git;
5. Android Studio;
6. Adobe Photoshop;
7. Adobe Illustrator;
8. XAMPP;
9. Visual Studio Code;

- 10. Visual Studio;
- 11. Microsoft Office;
- 12. IntelliJ IDEA Community Edition.

ГЛАВА 2 ОСНОВЫ АЛГОРИТМИЗАЦИИ И ПРОГРАММИРОВАНИЯ

2.1 Тестирование программных комплексов

В данной работе был проведён анализ и тестирование программных комплексов на языке программирования C#. Изначально язык программирования C# выпущенный вместе с Visual Studio .NET был очень схож с Java, он стремился быть простым, современным, объектно-ориентированным языком общего назначения [10].

Тестирование «Программного комплекса «Система тестирования»». (см. Таблица 1)

Таблица 1 «Сравнительные характеристики программ»

Имя файла	Плюсы	Минусы
Тест_Линейные программы (1)	В программе присутствует progress bar для отображения прогресса прохождения теста.	В полях со свободным ответом, при нажатии на пустое пространство, а затем повторное нажатие на поле ввода, программа стирает написанный пользователем код.
Курсовая	Программа сразу подсчитывает количество верно отвеченных вопросов и за счет, того, что отсутствует кнопка вернуться, программа не позволяет «накрутить» верных ответов.	Отсутствует кнопка, для возвращения к предыдущим вопросам. В вопросе 7 и 20 выделены правильные варианты ответа символом «*».
КР_Линейные программы_Тестирование_Боровинских П-27	В программе присутствует интерактив в виде «убегающих вариантов ответа». Также используется много	Нет нумерации вопросов. В вопросе 3 варианты ответа могут заслонить друг друга и перестать двигаться из-за чего нельзя выбрать тот варианта ответа,

	различных вариантов ответа для пользователя.	который был перекрыт. В следующем вопросе кнопка «Дальше» может попасть под текст.
Бобровская_П-28_курсовая	В программе используется много различных вариантов ответа для пользователя.	Отсутствует кнопка, для возвращения к предыдущим вопросам.
WindowsFormsApp11 (1)	В программе используется много различных вариантов ответа для пользователя.	Отсутствует кнопка, для возвращения к предыдущим вопросам. Большинство вопросов позволяют пройти не отвечая на них дальше, а некоторые нет. Пользователю не видно на каком он вопросе. На финальной форме нет кнопки, чтобы закрыть программу.
WindowsFormsApp2	Присутствует кнопка вернуться на предыдущие вопросы, в программе ведется подсчет пройденных вопросов для отображения прогресса прохождения теста.	Когда отвечаешь на вопрос и возвращаешься на него обратно, то введенные данные стираются.

2.2 Разработка программной оболочки

Для начала разработки программной оболочки было создано приложение Windows Forms. Потому-что в Windows Forms можно разрабатывать графически сложные приложения, которые просто развертывать, обновлять, и с которыми удобно работать как в автономном режиме, так и в сети [9].

Форма с именем «Form1» была переименована в «FormStart» для устранения путаницы и быстрой навигации во время разработки.

Дополнительно была создана еще одна форма с именем «Form2» и сразу была переименована в «FormTable».

Форма «FormStart» была изменена в размерах до 550 по ширине и 300 по высоте (см. рис. 2.1). На этой же форме было создано 3 элемента Label. Label с именем «Header_label» использовался, как заголовок, в нем был увеличен шрифт до 16 пунктов, а сам label был размещен сверху по центру формы. Элемент Label с именем «Combo_label» находился над элементом ComboBox для призыва пользователя к действию. Шрифт «Combo_label» был изменен до 12 пунктов. Цвет «error_label» был изменен с Control на DarkRed, а также перемещен в нижний левый угол. Была создана кнопка при клике, на которую пользователь мог перейти в выбранную им кафедру. В данный момент доступна только кафедра математики и информатики.

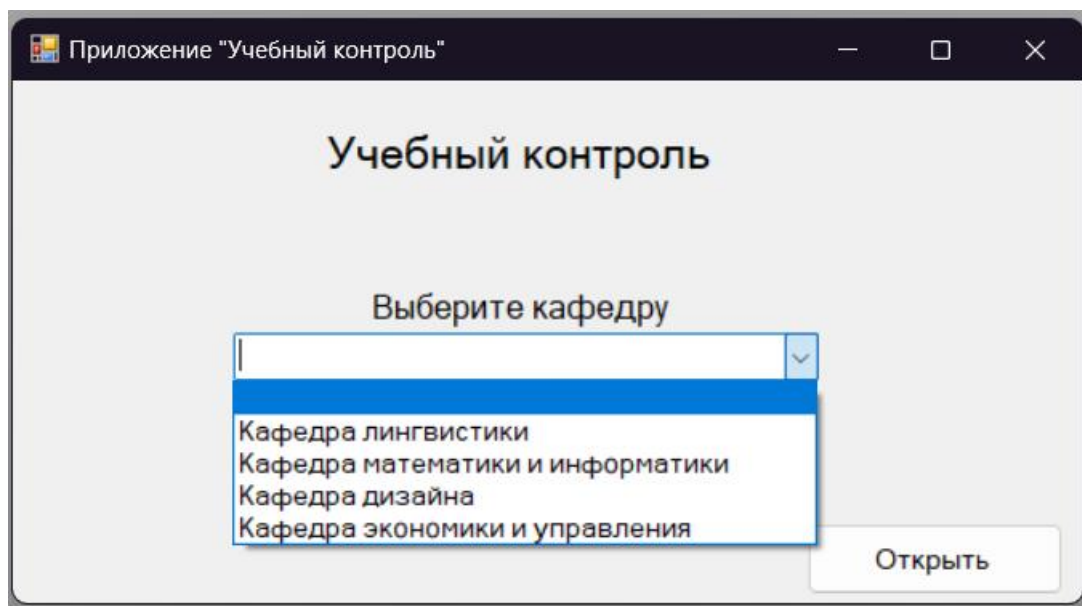


Рисунок 2.1 – Форма «FormStart»

Форма «FormLabel» была изменена в размерах до 950 по ширине и 715 по высоте (см. рис. 2.2). На этой же форме были использованы следующие элементы:

1. Label (8 шт.);
 - а. Фамилия.

- b. Имя.
 - c. Дата рождения.
 - d. Группа.
 - e. Оценка.
 - f. Кол-во пропусков.
 - g. info_output.
2. TextBox (2 шт.);
 3. CheckBox (1 шт.);
 4. ComboBox (1 шт.);
 5. NumericUpDown (2 шт.);
 6. Button (6 шт.);
 - a. Добавить студента.
 - b. Удалить студента.
 - c. Отчистить не сдавших отчет.
 - d. Отчистить таблицу.
 - e. Определить кол-во строк.
 - f. Заккрыть программу.
 7. DateTimePicker (1 шт.);
 8. DataGridView (1 шт.).

Семи элементам label был изменен размер шрифта до 10 пунктов, а элементу label «info_output» был изменен цвет на DarkRed, установлена жирность текста bold, размер шрифта 12 пунктов.

Шрифт в элементах TextBox был изменен до 8 пунктов, а также отредактирована ширина элементов.

Элементу CheckBox был изменен размер шрифта до 10 пунктов и изменен текст на «Сдал отчет по практике».

Элементу ComboBox был изменен размер шрифта до 8 пунктов и отредактирована ширина элемента.

Элементам NumericUpDown был изменен размер шрифта до 9 пунктов.

Элементам Button был изменен размер шрифта до 9 пунктов и изменен текст.

Элементу DateTimePicker был изменен размер шрифта до 10 пунктов и отредактирована ширина элемента.

Элемент DataGridView был изменен до 990 по ширине и 410 по высоте, добавлены и изменены по ширине 8 столбцов.

	Фамилия	Имя	Группа	Номер студента	Дата рождения	Отчет о прохождении практики	Оценка	Кол-во пропусков
--	---------	-----	--------	----------------	---------------	------------------------------	--------	------------------

Рисунок 2.2 – Форма «FormTable»

В программном комплексе присутствует возможность добавлять студента, определять кол-во строк, удалять студентов и отчищать таблицу.

При добавлении студента Фамилии и Имени используется textbox, для календаря DateTimePicker и checkbox, который показывает сдал ли студент отчет по практике. Чтобы пользователь мог ввести только реалистичную оценку или количество пропусков используются элементы NumericUpDown с

ограничением диапазона от 2 до 5 и кол-во пропусков от 0 до 400. В кодовой части программы при добавлении студента программа проверяет введены ли Фамилия, Имя, Группа, если нет, то выводит сообщение о необходимости заполнения данных полей.

По итогам работы были отработаны ранее полученные навыки с элементами Windows Forms, а также получена теоретическая информация о базах данных.

2.3 Индивидуальное задание. Проектирование и разработка программной оболочки риэлтерского агентства.

Для начала разработки программной оболочки «Реестр клиентов «РиелторПро» » было создано приложение Windows Forms. Форма с именем «Form1» была переименована в «RieltorPW» для удобства.

Форма «FormLabel» была изменена в размерах до 960 по ширине и 780 по высоте (см. рис. 2.3). На этой же форме были использованы следующие элементы:

1. Label (8 шт.);
 - a. Фамилия.
 - b. Имя.
 - c. Дата рождения.
 - d. Группа.
 - e. Оценка.
 - f. Кол-во пропусков.
 - g. info_output.
2. TextBox (2 шт.);
3. CheckBox (1 шт.);
4. ComboBox (1 шт.);
5. NumericUpDown (2 шт.);
6. Button (8 шт.);

- a. Добавить клиента.
 - b. Удалить клиента.
 - c. Отчистить не внесших предоплату.
 - d. Отчистить таблицу.
 - e. Определить кол-во клиентов.
 - f. Отчистить клиентов с рейтингом ниже 6
 - g. Отсортировать клиентов по бюджету
 - h. Закрыть программу.
7. DateTimePicker (1 шт.);
8. DataGridView (1 шт.).

Семи элементам label был изменен размер шрифта до 10 пунктов, а элементу label «info_output» был изменен цвет на DarkRed, установлена жирность текста bold, размер шрифта 12 пунктов.

Шрифт в элементах TextBox был изменен до 8 пунктов, а также отредактирована ширина элементов.

Элементу CheckBox был изменен размер шрифта до 10 пунктов и изменен текст на «Сдал отчет по практике».

Элементу ComboBox был изменен размер шрифта до 8 пунктов и отредактирована ширина элемента.

Элементам NumericUpDown был изменен размер шрифта до 9 пунктов.

Элементам Button был изменен размер шрифта до 9 пунктов и изменен текст.

Элементу DateTimePicker был изменен размер шрифта до 10 пунктов и отредактирована ширина элемента.

Элемент DataGridView был изменен до 990 по ширине и 410 по высоте, добавлены и изменены по ширине 8 столбцов.

Реестр клиентов "РиелторПро"

Реестр клиентов "РиелторПро"

Фамилия
 Цель

Имя
 Рейтинг

Дата регистрации
 Бюджет

☐ Внес предоплату за услуги

Добавить клиента

Номер клиента	Фамилия	Имя	Внес предоплату за услуги	Бюджет	Цель	Рейтинг	Дата регистрации

Отчистить не внесших предоплату
 Отчистить таблицу
 Определить кол-во клиентов
 Удалить клиента

Отчистить клиентов с рейтингом ниже 6
 Отсортировать клиентов по Бюджету

Рисунок 2.3 – Форма «RieltorPW»

В программном комплексе присутствует возможность добавлять клиентов, определять кол-во строк, удалять клиентов, отчищать таблицу, отчистить клиентов с рейтингом ниже 6 и сортировать клиентов по бюджету.

При добавлении клиента Фамилии и Имени используется textbox, для календаря DateTimePicker и Checkbox, который показывает внес ли студент предоплату за услуги. Чтобы пользователь мог ввести только рейтинг и бюджет используются элементы NumericUpDown рейтинг с ограничением диапазона от 1 до 10 и бюджет с ограничением диапазона от 10 000 до 1 000 000 и инкрементом равным 5000. В кодовой части программы при добавлении клиента программа проверяет введены ли Фамилия, Имя, Цель, если нет, то выводит сообщение о необходимости заполнения данных полей. При успешном добавлении клиента поля Фамилия и Имя очищаются.

При нажатии на кнопку «Отчистить клиентов с рейтингом ниже 6», то все клиенты с рейтингом ниже 6 будут удалены из таблицы.

При нажатии на кнопку «Отсортировать клиентов по бюджету», то все клиенты будут отсортированы по бюджету по принципу от большего к меньшему.

Подводя итоги, можно отметить, что была усовершенствована практика использования элементов Windows Forms, а также получена необходимая теоретическая база по работе с базами данных.

Вывод

В рамках освоения модуля по основам алгоритмизации и программирования был выполнен комплекс практических задач на языке C# с использованием фреймворка Windows Forms. Проведение сравнительного аудита готовых программных комплексов позволило выявить типичные интерфейсные уязвимости и определить правила создания UI/UX-решений. Разработка базовой системы «Учебный контроль» и создание итогового приложения «РиелторПро» для автоматизации учета клиентов закрепили практические навыки ООП, работы с динамическими структурами данных, а также механизмами валидации, фильтрации и сортировки информации.

ГЛАВА 3 ОПЕРАЦИОННЫЕ СИСТЕМЫ И СРЕДЫ

3.1 Введение в Kotlin и Android Studio

Kotlin – это статически типизированный, объектно-ориентированный язык программирования, работающий поверх Java Virtual Machine (JVM). Поддерживает как объектно-ориентированное, так и процедурное программирование [7].

Наша разработка будет проходить в программе Android Studio. Android – это портативная (сетевая) операционная система для коммуникаторов, планшетных компьютеров, электронных книжек, цифровых проигрывателей, наручных часов, нетбуков и смартфонов, основанная на ядре Linux. Android Studio, основанная на программном обеспечении IntelliJ IDEA от компании JetBrains, официальное средство разработки Android приложений. Данная среда разработки доступна для Windows, OS X и Linux [1].

3.2 Изучение жизненного цикла Android

Жизненный цикл Android – это последовательность состояний, через которые проходит компонент приложения (например, активность - основной компонент, представляющий экран или окно) от момента его создания до уничтожения или перехода в фоновый режим (см. рис. 3.1).

Когда пользователь переходит в ваше приложение, выходит из него и возвращается обратно, экземпляры Activity в вашем приложении переходят из одного состояния в другое на протяжении своего жизненного цикла. Класс Activity предоставляет ряд обратных вызовов, которые сообщают Activity об изменении состояния или о том, что система создает, останавливает или возобновляет действие или уничтожает процесс, в котором находится действие [3].

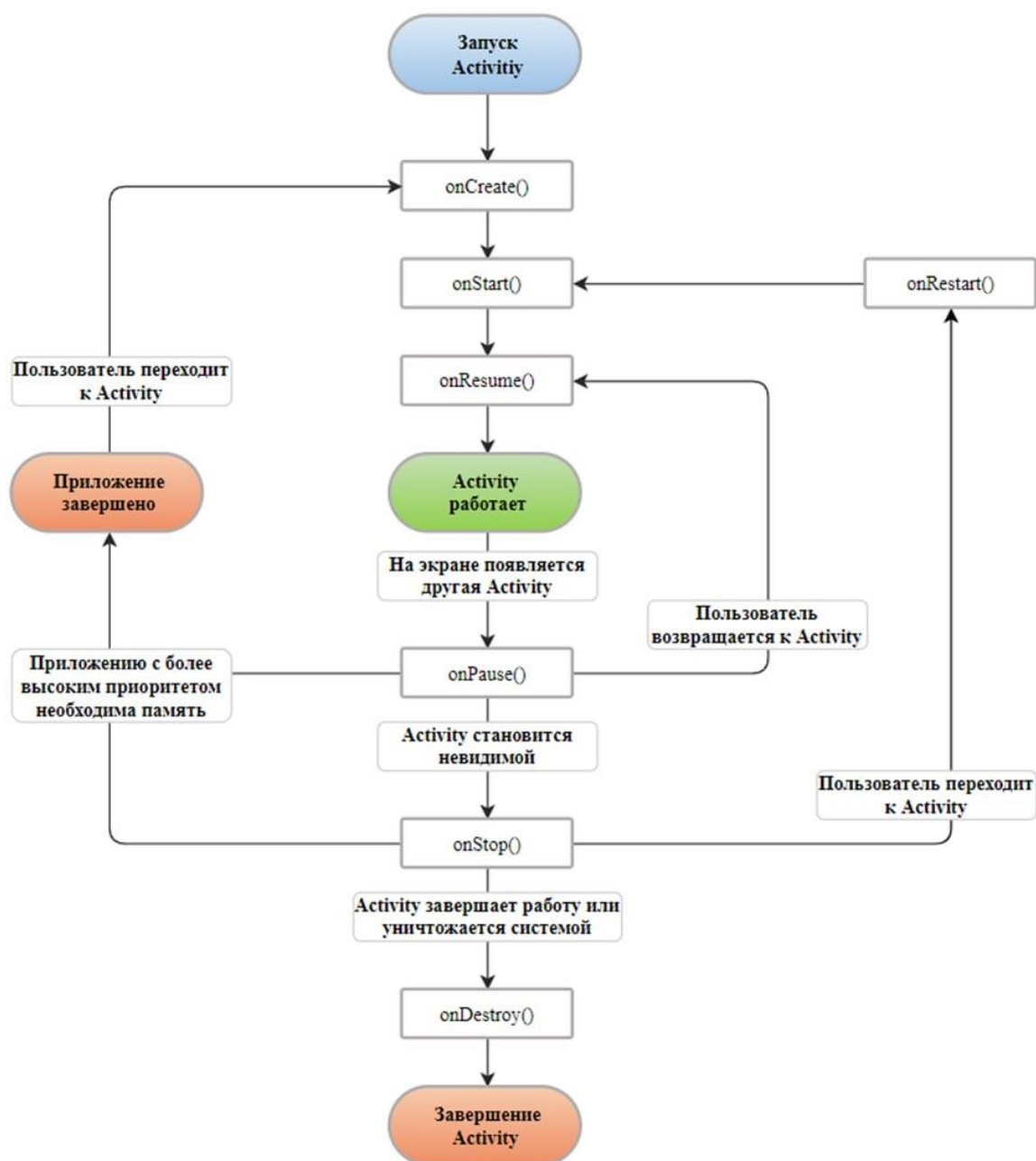


Рисунок 3.1 – Работа жизненного цикла Android

3.3 Верстка на языке Kotlin

Начальным этапом проектирования и разработки мобильных интерфейсов стало детальное ознакомление со средой разработки Android Studio. В качестве базового шаблона для создания современных мобильных приложений было выбрано решение «Empty Activity» на базе фреймворка декларативного пользовательского интерфейса Jetpack Compose.

Автоматически сгенерированная структура проекта включает в себя комплекс конфигурационных файлов, манифест приложения, а также предустановленные каталоги ресурсов (включая системные цветовые палитры `ui.theme`, шрифтовые стили и стандартные векторные иконки).

Центральным компонентом и точкой входа в приложение является файл `MainActivity.kt`. Данный класс наследуется от `ComponentActivity` и содержит переопределенный метод жизненного цикла `onCreate()`. Внутри метода вызывается функция `setContent()`, которая инициализирует блок конфигурации, устанавливает начальную тему оформления и монтирует корневую композитную функцию, выступающую в роли базиса всего пользовательского интерфейса. На начальных этапах изучения мобильной разработки весь ключевой программный код и UI-компоненты фокусируются непосредственно в данном корневом файле.

В рамках фреймворка Jetpack Compose для проектирования визуальной части применяется декларативный подход, реализуемый посредством композитных функций. Композитная функция представляет собой программный модуль, который принимает входные аргументы (состояние, данные) и трансформирует их в описание структуры экранного интерфейса. Для инициализации и корректной компиляции такой функции средой разработки перед ее объявлением в обязательном порядке указывается аннотация `@Composable`.

Основным инструментарием конструирования UI выступают стандартные графические элементы и специализированные контейнеры компоновки (`Layouts`), отвечающие за пространственное расположение объектов на экране мобильного устройства.

Все базовые компоненты и контейнеры обладают набором переопределяемых параметров, с помощью которых осуществляется наполнение контентом и их точечная конфигурация. Ключевым и наиболее универсальным инструментом настройки выступает модификатор `Modifier`. Посредством вызова его цепочечных методов реализуется управление

геометрическими размерами элементов, настройка внутренних и внешних отступов, управление границами, скруглением углов, а также привязка обработчиков событий.

Для оптимизации процесса разработки и исключения необходимости постоянной ресурсоемкой компиляции приложения на физическом устройстве или эмуляторе применяется аннотация `@Preview`.

Комплексное индивидуальное задание по мобильной разработке было разделено на три последовательных этапа: от построения простейших изолированных контейнеров до создания многостраничного приложения с динамической системой навигации.

Первый этап был ориентирован на выявление и закрепление навыков базовой работы со стандартными UI-компонентами Jetpack Compose, управление текстовыми метриками, размерами шрифтов и пространственными отступами. Задание включало реализацию двух практических задач:

1. Реализация функции, последовательно размещающей графическое изображение (Image), текстовый заголовок (Text) и блок описания с явным указанием параметров масштабирования и модификаторов отступов.

2. Создание интерфейса, в котором контейнер верхнего уровня Box занимает все доступное пространство экрана и выравнивает вложенное содержимое строго по центру. Внутри контейнера с помощью вертикального распределения Column реализовано симметричное позиционирование системной иконки Icon (размером 100.dp) и сопутствующей текстовой метки «Контакты».

Второй этап был направлен на освоение интерактивных элементов взаимодействия с пользователем и организацию табличных интерфейсов. В ходе работы были решены следующие задачи:

- Проектирование форм сбора данных: разработка интерфейсов на базе компонентов текстового ввода TextField (и его модификации

OutlinedTextField) для ввода пользовательских строк и управляющих кнопок Button.

- Верстка структурированных списков (карточек): на основе переданных классов данных была спроектирована карточка элемента (компонент Card), содержащая изображение, текстовые поля названия и стоимости. Карточки были организованы в двухколоночную адаптивную сетку с помощью контейнера LazyVerticalGrid с параметром GridCells.Fixed(2), реализующего ленивую загрузку и эффективную оптимизацию памяти при скроллинге.

- Повторная интеграция компонентов: четвертая задача этапа была нацелена на закрепление материала и дублировала логику центрирования базовых иконок и текстовых блоков, отработанную на первом этапе.

Финальным и наиболее сложным этапом работы стала верстка полноценного мобильного приложения (каталога игр) с внедрением механики безопасной навигации между экранами на базе архитектурного компонента NavHostController.

В рамках данного приложения была реализована следующая программная логика:

1. Главный экран отображает вертикальный список элементов, контент для которого поставляется из заготовленного списка объектов. Каждая строка списка представляет собой интерактивную карточку игрового приложения, содержащую превью-изображение, название игры, жанровую принадлежность и ценовой маркер.

2. Экран с информацией: переход на данный экран осуществляется по клику на карточку из списка. Навигатор динамически передает уникальный идентификатор объекта (game.id) через строку маршрута.

3. Контроллер навигации: связывает экраны в единую систему с помощью конструкции NavHost, определяя Screen.Main.route как стартовую точку приложения и обрабатывая асинхронный перенос состояний и аргументов между композитными экранами по запросу пользователя.

Вывод

Изучение операционных систем и сред позволило освоить современные технологии разработки мобильного программного обеспечения для ОС Android в среде Android Studio. В ходе работы был закреплён теоретический базис программирования на языке Kotlin и детально изучена механика состояний жизненного цикла активности. Практическая реализация многостраничного приложения на базе фреймворка Jetpack Compose позволила освоить передовой декларативный подход к верстке, механизмы гибкого управления компонентами с помощью модификаторов, а также принципы организации ленивых списков и построения безопасной навигации между экранами.

ГЛАВА 4 БАЗЫ ДАННЫХ

4.1 Инфологическое проектирование баз данных

База данных — организованная в соответствии с определенными правилами и поддерживаемая в памяти компьютера совокупность данных, характеризующая актуальное состояние некоторой предметной области и используемая для удовлетворения информационных потребностей пользователей [6].

На вводном занятии по базам данных были получены навыки для дальнейшей работы с базами данных. Были даны определения для понятий База данных, СУБД, первичный и вторичный ключ, разобраны типы баз данных и выявлены основные функции СУБД.

СУБД (система управления базой данных) — это комплекс программных и языковых средств, предназначенный для создания баз данных, а также управления, поиска и изменения хранящейся в них информации.

Функции СУБД:

- Создание базы данных
- Интерфейс манипулирования данными
- Управления данными во внешней и оперативной памяти
- Обеспечить логической и физической целостностью данных
- Защита логической и физической целостности данных
- Разграничения прав доступа к информации определений ролей и их привилегий при работе с базами данных.
- Синхронизация работы нескольких пользователей

Сущность — это класс однотипных объектов, информация о которых должна быть учтена в модели. Каждая сущность должна иметь наименование, выраженное существительным в единственном числе. Каждая сущность в модели изображается в виде прямоугольника с наименованием.

У каждой сущности есть атрибуты, которые являются её характеристиками и свойствами. Например, у сущности «Ученик» есть свойства ФИО, адрес, тел. номер, дата рождения и т. д.

Между сущностями существует некая ассоциация, которая называется связью. Одна сущность может быть связана с другой сущностью или сама с собой. Графически связь изображается линией, соединяющей две сущности. Каждая связь характеризуется именем, типом, классом принадлежности и направлением (Рис.1). Различают четыре типа связи: «один к одному» (1:1); «один ко многим» (1:M); «многие к одному» (M:1) «многие ко многим» (M:M) [5].

1. Сущности:
 1. Склад ([PK] Код склада, Наименование, Адрес);
 2. Товар ([PK] Артикул, Название, Описание, Цена, Срок годности (дней));
 3. Поставщик ([PK] ИНН, Адрес, Название, Среднее время доставки, Страна);
 4. Пользователь ([PK] Код пользователя, Фамилия, Имя, Отчество, Дата рождения, Пол);
 5. Сотрудник ([PK] [FK] Код сотрудника, Рейтинг, Тип рабочего договора);
 6. Клиент ([PK] [FK] Код клиента, Логин, Пароль, Email, Телефон);
 7. Отдел ([PK] Код отдела, Название, Дата основания);
 8. Должность ([PK] Код должности, Название, Оклад, Должностная инструкция);
 9. Ячейка ([PK] Номер ячейки, Вместимость, Стеллаж, Полка);
 10. Поставка ([PK] Код поставки, [FK] Код склада, [FK] ИНН поставщика, Дата поставки, Итоговая стоимость);
 11. Товары в поставке ([PK] Код записи, [FK] Артикул, [FK] Код поставки, Количество, Цена);

12. Товары на складах ([PK] Код записи, [FK], Код записи, [FK] Артикул товара, [FK] Номер ячейки, Количество);

13. Заказ ([PK] Код заказа, [FK] Код клиента, [FK] Код склада, Дата, Статус);

14. Товары в заказе ([PK] Код записи, [FK] Код склада, [FK] Артикул товара, Количество, Цена).

2. Определение связей (выделить 2 связываемые сущности, определить степень связи, выделить главную и дочернюю сущности, добавить внешний ключ в дочернюю сущность. Определить класс принадлежности связи):

Связь «один-к-одному» между сущностями «Пользователь» и «Сотрудник» образована парой первичного ключа «Код пользователя» из главной сущности «Пользователь» и внешнего ключа «Код сотрудника» из дочерней сущности «Сотрудник»

1. «Пользователь» 1 → 1 «Клиент»
2. «Отдел» 1 → М «Сотрудник»
3. «Должность» 1 → М «Сотрудник»
4. Связь «Товар» М → М «Склад» была разбита на 2 связи:
 - а. «Товар» 1 → М «Товары на складах»
 - б. «Склад» 1 → М «Товары на складах»
5. Связь «Склад» М → М «Поставщик» была разбита на 2 связи:
 - а. «Склад» 1 → М «Поставка»
 - б. «Поставщик» 1 → М «Поставка»
6. Связь «Поставка» М → М «Товар» была разбита на 2 связи:
 - а. «Поставка» 1 → М «Товары в поставке»
 - б. «Товар» 1 → М «Товары в поставке»
7. Связь «Склад» М → М «Ячейка» была разбита на 2 связи:
 - а. «Склад» 1 → М «Товары на складах»
 - б. «Ячейка» 1 → М «Товары на складах»
8. Связь «Клиент» М → М «Склад» была разбита на 2 связи:

- a. «Клиент» 1 → М «Заказ»
 - b. «Склад» 1 → М «Заказ»
9. Связь «Заказ» М → М «Товар» была разбита на 2 связи:
- a. «Заказ» 1 → М «Товары в заказе»
 - b. «Товар» 1 → М «Товары в заказе»
3. Определение ограничений целостности:
1. Товар.Цена > 0;
 2. Товар.Срок_годности(дней) > 0;
 3. Поставщик.ИНН соответствует маске «999999999999»;
 4. Пользователь.Дата_рождения <= текущая_дата
 5. Пользователь.Пол входит в набор [Мужской, Женский, ...]
 6. Сотрудник.Рейтинг между 0 и 10 (оператор between в SQL)
 7. Сотрудник.Тип_рабочего_договора входит в набор [Штатный, По договору]
 8. Клиент.email валидный адрес электронной почты
 9. Клиент.Телефон соответствует маске «+9 (999) 999-99-99»
 10. Отдел.Дата_основания <= Текущая дата
 11. Должность.Оклад > 0
 12. Ячейка.Вместимость > 0
 13. Ячейка.Стеллаж > 0
 14. Ячейка.Полка > 0
 15. Поставка.Итоговая_стоимость > 0 & Поставка.Итоговая_стоимость = СУММА(Товар_в_поставке.Количество * Товар_в_поставке.Цена)
 16. Товары_в_поставке.Количество > 0
 17. Товары_в_поставке.Цена > 0
 18. Товары_на_Складах >= 0
 19. Заказ.Дата <= текущая_дата
 20. Заказ.Статус входит в набор [Собирается, В пути, Готов к выдаче]
 21. Товары_в_заказе.Количество > 0

22. Товары_в_заказе.Цена > 0

4.2 Логическое проектирование баз данных

Логическое (дatalogическое) проектирование – это отображение концептуальной модели в дatalogическую модель данных.

Так как подавляющее большинство современных СУБД реляционные, то в дальнейшем при проектировании баз данных в качестве дatalogической модели будем использовать реляционную модель.

Как известно, реляционная модель представляет собой набор таблиц (обычно с указанием ключевых полей) и связей между таблицами. Если концептуальная модель построена в виде диаграммы «сущность – связь», то логическое проектирование состоит в построении таблиц по определённым формализованным правилам, а также в нормализации этих таблиц [2].

Следующим шагом в проектировании баз данных является логическое проектирование. На этом этапе необходимо выполнить заключительный этап.

Выбор СУБД – Критерии:

1. Функциональные возможности.
2. Особенности разработки приложений.
3. Производительность.
4. Надежность.
5. Требования к рабочей среде.

Требование к логическому проектированию:

1. Минимальность первичных ключей.
2. Надежность данных.
3. Производительность системы.
4. Гибкость структуры БД.
5. Актуальность данных.

6. Непротиворечивость данных.

4.3 Индивидуальное задание. Инфологическое и логическое проектирования баз данных

На данном этапе была создана концептуальная схема базы данных на основе технического задания, в котором были представлены задачи про филиалы компании, складской учет и игровую статистику. Проектирование шло по стандартному алгоритму:

1. **Предметная область.** Сначала внимательно изучались исходные условия, чтобы понять, какие объекты вообще требовалось отслеживать и как они между собой были связаны.

2. **Выделение сущностей.** Выделялись главные объекты, по которым распределялись соответствующие свойства. Например, в итоге получились такие сущности, как «Филиал», «Сотрудник», «Игрок», «Уровень» и «Валюта».

3. **Назначение ключей.** Для каждой сущности выбирался уникальный идентификатор – первичный ключ (РК), исключающий путаницу в записях. Для сотрудников использовался «Номер личного дела», для геймеров – «ID игрока».

4. **Связывание таблиц.** Настраивались логические связи. Простые отношения типа «один ко многим» (1:N) создавались напрямую (например, в одном филиале числилось множество сотрудников). Сложные связи типа «многие ко многим» (M:N) разбивались через промежуточные таблицы вроде «Прогресс» или «Покупка валюты».

5. **Прописывание ограничений.** Фиксировались правила для сохранения целостности данных: настраивалась ссылочная целостность (блокировалось удаление ветки, если к ней были привязаны люди) и указывалось, какие поля обязательно должны были быть заполнены.

На этапе логического проектирования абстрактная схема переводилась в нормальный реляционный вид, а таблицы доводились до полной готовности. Процесс выглядел следующим образом:

1. **Создание реляционной структуры.** Сущности переносились в обычные таблицы, а связи 1:N фиксировались через внешние ключи (FK) в подчиненных таблицах.

2. **Нормализация.** С целью избавления от дублирования данных и исключения косяков при обновлении, таблицы последовательно прогонялись по нормальным формам:

- **1НФ (Первая нормальная форма):** разбирались все сложные строки, чтобы в каждой ячейке находилось строго одно атомарное значение.

- **2НФ (Вторая нормальная форма):** проверялось соответствие всех таблиц требованиям 1НФ, а также полная зависимость неключевых полей от главного составного ключа.

- **3НФ (Третья нормальная форма):** устранялись транзитивные зависимости. Все повторяющиеся данные выносились в отдельные справочники (таким образом появились таблицы «Подразделения» и «Должности»).

- **4НФ (Четвертая нормальная форма):** в шестом задании со студентами устранялись многозначные зависимости – для этого «Курсы» и «Хобби» разделялись по разным таблицам, поскольку они вообще никак между собой не были связаны.

Вывод

При изучении баз данных были успешно освоены ключевые этапы проектирования реляционных информационных хранилищ. На инфологическом этапе был проведен системный анализ предметной области складского учета, в ходе которого определены базовые сущности, декомпозированы сложные многозначные связи и сформулированы ограничения целостности данных. На этапе логического проектирования

осуществлен успешный переход к даталогической реляционной модели, определены критерии выбора эффективных СУБД и изучены требования к нормализации таблиц, что обеспечило готовность проекта к физической реализации.

ГЛАВА 5 ЛИНЕЙНОЕ ПРОГРАММИРОВАНИЕ

5.1 Экономико-математическая модель

Экономико-математическая модель – достаточно точное описание исследуемого экономического процесса или объекта с помощью математического аппарата (различного рода функций, уравнений, систем уравнений и неравенств и т.п.).

Переменными ЗМП являются x_i , которые полностью характеризуют экономический процесс.

Система ограничений – это система уравнений или неравенств, которым удовлетворяют x_i и которые следуют из ограниченности ресурсов или т.п. условий.

Целевой функцией называется функция переменных задачи экстремум которой надо найти. Она характеризует качество выполнения задачи.

Если целевая функция и система ограничений линейны, то ЗМП (задача математического программирования) называется задачей линейного программирования.

5.2 Индивидуальное задание. Построение экономико-математической модели

В данной работе были применены знания по построению экономико-математической модели.

Условие задачи А. Фирма производит одежду двух видов: платья и костюмы. В неделю фирма продает 600 изделий. Для каждого платья требуется 3м. полотна, а для костюма 5м. Фирма в неделю получает 1200м. полотна. Для шитья 1 платья требуется 30 минут, а для шитья костюма 40 минут. Оборудование может использоваться не больше 80 часов в неделю.

Если прибыль от продаж платья – 50 \$, то от костюма – 85 \$. Сколько изделий надо выпускать в неделю для получения максимальной прибыли? (см. рис. 5.2.1-5.2.2).

Задачи линейного прогн-я. Практика 2 курс

Задача А - 5:

Фирма производит одежду двух видов: платья и костюмы. В неделю фирма продает 600 изделий. Для каждого платья требуется 3 м полотна, а для костюма 5 м. Фирма в неделю получает 1200 м полотна. Для шитья 1 платья требуется 30 минут, а для шитья костюма 40 минут. Оборудование может использоваться не больше 80 часов в неделю.

Если прибыль от продаж платья – 50 \$, то от костюма – 85 \$. Сколько изделий надо выпускать в неделю для получения максимальной прибыли?

Рисунок 5.2.1 – Лист 1 «Построение ЭММ» Задача А

Одежда	Время	Полотно	Всего полотна	Цена
ПЛАТЬЕ	30 м	3 м	1200 м	50 \$
КОСТЮМ	40 м	5 м		85 \$

x_1 – ПЛАТЬЕ 1 шт.
 x_2 – КОСТЮМ 1 шт.
 функция цели: $f(x) = 50x_1 + 85x_2 \rightarrow \max$
 Ограничения:
 $x_1, x_2 \geq 0$
 $30x_1 + 40x_2 \leq 80 \cdot 60$
 $3x_1 + 5x_2 \leq 1200$
 $x_1 \geq 0$

Рисунок 5.2.2 – Лист 2 «Построение ЭММ» Задача А

Условие задачи Б. Прядильная фабрика для производства 2 видов пряжи использует три типа сырья – чистую шерсть, капрон и акрил. Требуется составить план производства пряжи с целью максимизации суммарной прибыли. (см. рис. 5.2.3).

Задача Б-6:

Прядильная фабрика для производства 2 видов пряжи использует три типа сырья – чистую шерсть, капрон и акрил.

Тип сырья	Нормы расхода на 1т		кол-во сырья
	вид 1	вид 2	
Шерсть	0,5	0,2	600
Капрон	0,1	0,4	620
Акрил	0,4	0,2	500
прибыль от реализации пряжи	1100	900	

Требуется составить план производства пряжи с целью максимизации суммарной прибыли.

x_1 – вид 1 А – шерсть В – капрон
 x_2 – вид 2 С – акрил

функция цели: $f(x) = 1100x_1 + 900x_2 \rightarrow \max$

Ограничения:

по сырью А: $0,5x_1 + 0,2x_2 \leq 600$
по сырью В: $0,1x_1 + 0,4x_2 \leq 620$
по сырью С: $0,4x_1 + 0,2x_2 \leq 500$

Рисунок 5.2.3 – Лист 3 «Построение ЭММ» Задача Б

5.3 Графический метод решения ЗЛП

Задача линейного программирования - найти экстремум целевой функции задачи и соответствующие ему решения (переменные) при условии, что эти переменные удовлетворяют системе ограничений и условиям неотрицательности.

Допустимым решением (планом) ЗЛП называется любой n-мерный вектор X , удовлетворяющий системе ограничений и условиям неотрицательности.

Множество допустимых решений задачи образует область допустимых решений задачи.

Оптимальным решением (планом) ЗЛП называется такое допустимое решение задачи, при котором целевая функция достигает экстремума.

Алгоритм решения ЗЛП с двумя переменными графическим методом:

1. По неравенствам ограничений строим многоугольник допустимых значений
2. Строим вектор нормали целевой функции $n\{c_1; c_2\}$ с приложением в начале координат
3. Перпендикулярно вектору нормали строим одну из линий уровня.
4. Перемещаем линию уровня по предельного положения. (точки или отрезка).
 - если предельное положение в точке, то находим ее координаты и вычисляем значение целевой функции;
 - если предельное положение на отрезке, то значение целевой функции = свободному члену в уравнении прямой этого отрезка.

5.4 Индивидуальное задание. Решение ЗЛП графическим методом

Предприятие предполагает выпускать два вида продукции A_1 и A_2 , для производства которых используется сырье трех видов. Производство обеспечено сырьем каждого вида в количествах: b_1, b_2, b_3 кг. На изготовление единицы изделия A_1 требуются затратить сырья каждого вида a_{11}, a_{21}, a_{31} кг, соответственно, а для единицы изделия A_2 - a_{12}, a_{22}, a_{32} кг, соответственно. Выручка от реализации единицы изделия A_1 составляет c_1 д.ед., для A_2 составляет c_2 д.ед.

Требуется составить план производства изделий A_1 и A_2 , обеспечивающих максимальную прибыль предприятия от реализации готовой продукции (см. рис. 5.3.1-5.3.3).

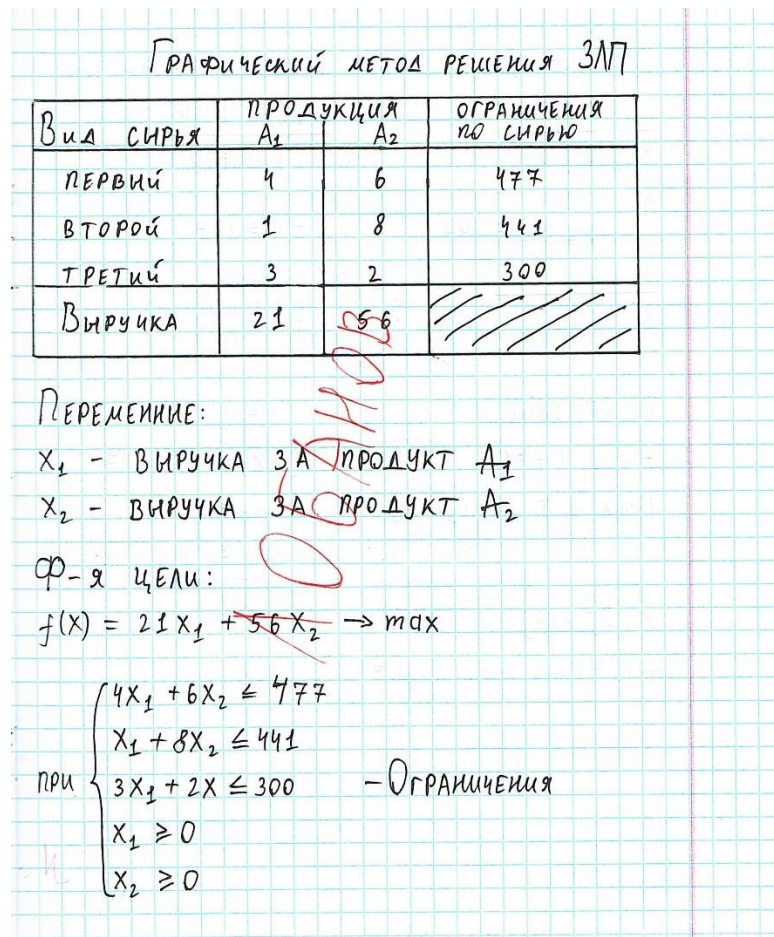


Рисунок 5.3.1 – Лист 1 «Решение ЗЛП графическим методом»

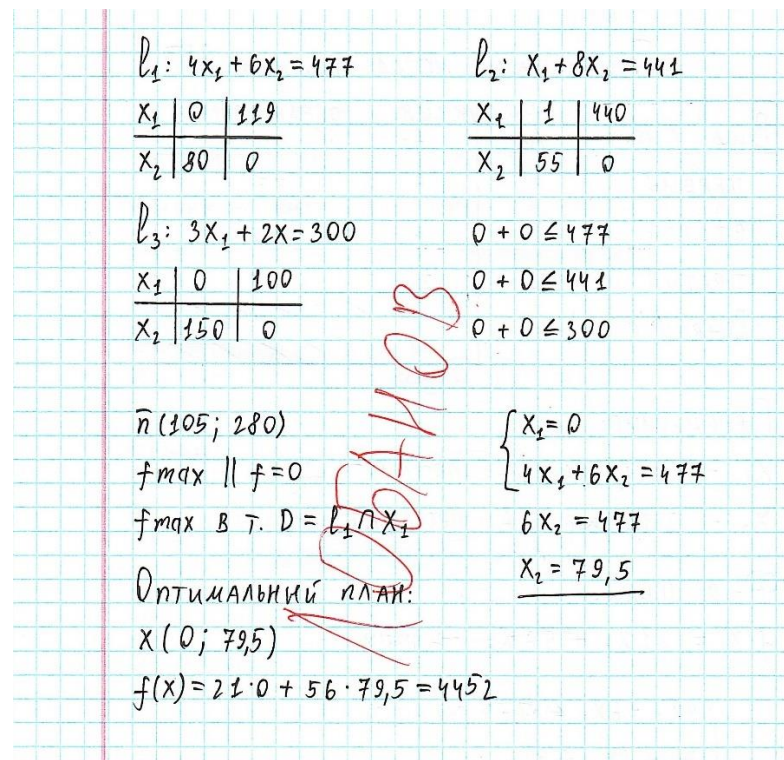


Рисунок 5.3.2 – Лист 2 «Решение ЗЛП графическим методом»

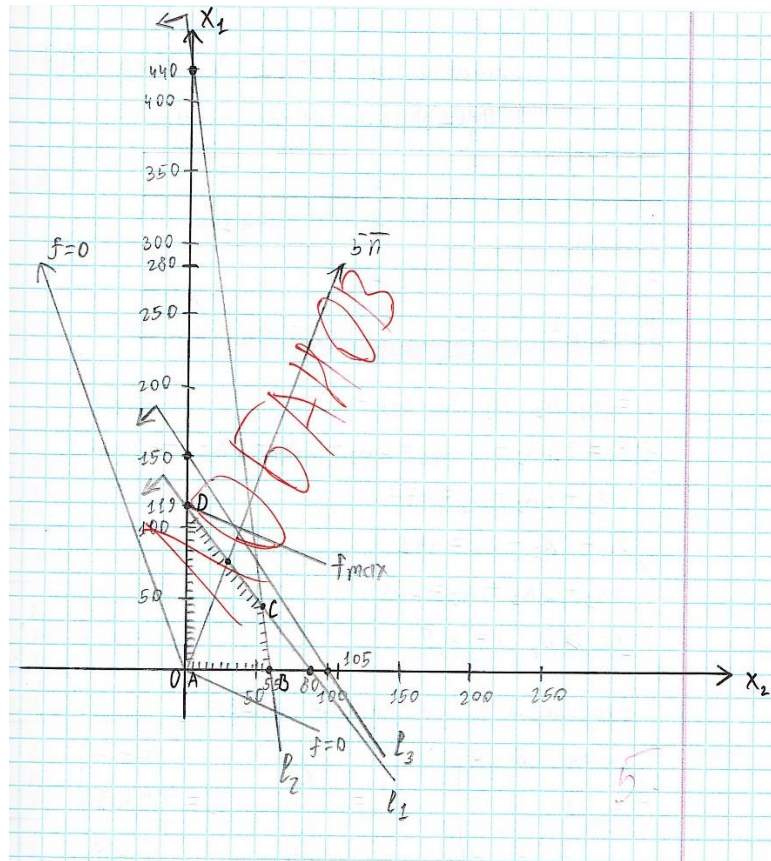


Рисунок 5.3.3 – Лист 3 «Решение ЗЛП графическим методом»

5.5 Симплексный метод решения ЗЛП

Если все неравенства ограничений содержат только знак $\leq$, то задача называется стандартной. (знак $\geq$ можно изменить, умножив обе части неравенства на (-1)).

Если все ограничения являются уравнениями и все переменные удовлетворяют условиям не отрицательности, то ЗЛП называется канонической.

Решение, соответствующее одной из вершин многоугольника допустимых решений называют опорным решением, а саму вершину - опорной точкой.1

Графический метод решения ЗЛП предполагает, что достаточно перебрать все опорные точки и выбрать из них ту, где функция цели будет максимальной.

Симплексный метод решения ЗЛП заключается в выборе некоторой части опорных решений по определенному правилу, обеспечивающему последовательное улучшение плана и достижения экстремума функции цели.

Задачи симплекс-метода:

1. Указать способ нахождения начального опорного решения.
2. Указать способ перехода от одного опорного решения к другому, при котором значение целевой функции будет ближе к оптимальному.
3. Указать критерии, которые позволят своевременно прекратить перебор решений, получив оптимальное или сделать заключение об его отсутствии.

Симплекс-метод использует симплекс-таблицы, которые строятся как расширенная матрица канонической задачи.

Составим таблицу 1 для начального опорного решения (см. Таблица 2)

Таблица 2 «Начальное опорное решение»

Таблица 1	x_1	x_2	x_3	x_4	x_5	b_i
x_3	2	1	1	0	0	400
x_4	3	4	0	1	1	900
x_5	1	3	0	0	1	600
f	-60	-40	0	0	0	0

Элементы первого столбца указывают базисные переменные и функцию цели.

Элементы первой строки все переменные и b_i .

Нижняя строка f называется индексной - здесь указаны коэффициенты переменных в функции цели.

5.5 Индивидуальное задание. Решение ЗЛП симплексным методом

Предприятие предполагает выпускать два вида продукции A_1 и A_2 , для производства которых используется сырье трех видов. Производство обеспечено сырьем каждого вида в количествах: b_1, b_2, b_3 кг. На изготовление единицы изделия A_1 требуются затратить сырья каждого вида a_{11}, a_{21}, a_{31} кг, соответственно, а для единицы изделия A_2 - a_{12}, a_{22}, a_{32} кг, соответственно. Выручка от реализации единицы изделия A_1 составляет c_1 д.ед., для A_2 составляет c_2 д.ед.

Требуется составить план производства изделий A_1 и A_2 , обеспечивающих максимальную прибыль предприятия от реализации готовой продукции (см. рис. 5.5.1-5.5.3).

Симплексный метод решения ЗЛП

Вид сырья	продукция A_1	продукция A_2	ограничения по сырью
первый	4	6	477
второй	1	8	441
третий	3	2	300
Выручка	21	56	

$x = (x_1, x_2, x_3, x_4, x_5) - ?$
 $f(x) = 21x_1 + 56x_2 + 0x_3 + 0x_4 + 0x_5 \rightarrow \max$

$$\begin{cases} 4x_1 + 6x_2 + x_3 = 477 \\ x_1 + 8x_2 + x_4 = 441 \\ 3x_1 + 2x_2 + x_5 = 300 \\ x_i \geq 0, i = 1, 5 \end{cases}$$

Перепишем в матричном виде:

$$A = \begin{pmatrix} 4 & 6 & 0 \\ 1 & 8 & 0 \\ 3 & 2 & 0 \end{pmatrix} \quad X = \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \end{pmatrix} \quad B = \begin{pmatrix} 477 \\ 441 \\ 300 \end{pmatrix}$$

Рисунок 5.5.1 – Лист 1 «Решение ЗЛП симплексным методом»

Ф-я цели в НЕКВНОМ ВИДЕ

$$f - 21x_1 - 56x_2 = 0$$

$$x_1 = x_2 = 0, \quad x_3 = 477$$

$$x_4 = 441, \quad f = 0$$

$$x_5 = 300$$

1 ОПОРНОЕ РЕШЕНИЕ

	x_1	x_2	x_3	x_4	x_5	B_i	B_i/a_{ij}
x_3	4	6	1	0	0	477	$477/6 = 79,5$
x_4	1	8	0	1	0	441	$441/8 = 55,1$ min
x_5	3	2	0	0	1	300	$300/2 = 150$
f	-21	-56	0	0	0	0	
x_3	$13/4$	0	1	$-3/4$	0	$585/4$	$159/2 : 13/4 = 45$ min
x_4	$1/8$	1	0	$1/8$	0	$441/8$	$441/8 : 1/8 = 441$
x_5	$11/4$	0	0	$-1/4$	1	$783/4$	$759/4 : 11/4 = 69$
f	-14	0	0	7	0	3087	
x_1	1	0	$1/3$	$-2/3$	0	45	
x_2	0	1	$-1/6$	$2/3$	0	$441/3$	
x_5	0	0	$-1/3$	$5/3$	1	66	
f	0	0	$56/3$	$49/3$	0	3717	

Рисунок 5.5.2 – Лист 2 «Решение ЗЛП симплексным методом»

Вывод:

При производстве

$$x_1 = 21 \quad \text{и} \quad x_2 = 56$$

$$f_{\max} = 3717$$

Рисунок 5.5.3 – Лист 3 «Решение ЗЛП симплексным методом»

5.6 Решение ЗЛП двойственным методом

Каждой ЗЛП соответствует другая задача, называемая двойственной к исходной. Предполагаем, что некоторая организация решила закупить ресурсы предприятия и необходимо установить оптимальные цены на ресурсы (оценки ресурсов), чтобы затраты на ресурсы были минимальными.

Составим и сформулируем двойственную задачу к задаче планирования производства:

Таблица 2 «Составление двойственной задачи»

Задача 1(исходная)	Задача 2 (двойственная)
Найти $X=(x_1; x_2)$, чтобы $F = c_1 x_1 + c_2 x_2 \rightarrow \max$ при ограничениях $\begin{cases} a_{11}x_1 + a_{12}x_2 \leq b_1 \\ a_{21}x_1 + a_{22}x_2 \leq b_2 \\ \dots \\ a_{m1}x_1 + a_{m2}x_2 \leq b_m \\ x_1 \geq 0 \quad x_2 \geq 0 \end{cases}$ Составить такой план выпуска продукции $X=(x_1; x_2)$, при котором прибыль от реализации продукции будет максимальной и потребление ресурсов по каждому виду продукции не превосходит имеющихся запасов	Найти $Y=(y_1; y_2; \dots y_m)$, чтобы $Z = b_1 y_1 + b_2 y_2 + \dots b_m y_m \rightarrow \min$ при ограничениях $\begin{cases} a_{11}y_1 + \dots + a_{m1}y_m \geq c_1 \\ a_{12}y_1 + \dots + a_{m2}y_m \geq c_2 \\ y_1 \geq 0 \dots y_m \geq 0 \end{cases}$ Найти такой набор оценок ресурсов $Y=(y_1; y_2; \dots y_m)$, при котором общие затраты на ресурсы будут минимальными и затраты на ресурсы при производстве каждого вида продукции будут не менее прибыли от реализации этой продукции

Теорема 1. Если одна из двойственных задач имеет оптимальное решение, то и другая имеет оптимальное решение, причем $F_{\max} = Z_{\min}$ при одном и том же оптимальном плане.

Теорема 2. Значение двойственных оценок $Y=(y_1; y_2; \dots y_m)$ можно найти в индексной строке симплекс-таблице оптимальной для исходной задачи в столбцах, соответствующих дополнительным введенным переменным.

5.7 Индивидуальное задание. Решение ЗЛП двойственным методом

Предприятие предполагает выпускать два вида продукции A_1 и A_2 , для производства которых используется сырье трех видов. Производство обеспечено сырьем каждого вида в количествах: b_1, b_2, b_3 кг. На изготовление

единицы изделия A_1 требуются затратить сырья каждого вида a_{11} , a_{21} , a_{31} кг, соответственно, а для единицы изделия A_2 - a_{12} , a_{22} , a_{32} кг, соответственно. Выручка от реализации единицы изделия A_1 составляет c_1 д.ед., для A_2 составляет c_2 д.ед.

Требуется составить двойственный план производства изделий A_1 и A_2 , обеспечивающих максимальную прибыль предприятия от реализации готовой продукции (см. рис. 5.7.1-5.7.2).

Двойственная задача

$$f(x) = 21x_1 + 56x_2 \rightarrow \max$$

$$\begin{cases} 4x_1 + 6x_2 \leq 477 \\ x_1 + 8x_2 \leq 441 \\ 3x_1 + 2x_2 \leq 300 \\ x_i \geq 0 \end{cases}$$

$$A_1 = \begin{pmatrix} 4 & 6 & 477 \\ 1 & 8 & 441 \\ 3 & 2 & 300 \\ 21 & 56 & f \end{pmatrix} \Rightarrow A_2 = \begin{pmatrix} 4 & 1 & 3 & 21 \\ 6 & 8 & 2 & 56 \\ 477 & 441 & 300 & Z \end{pmatrix}$$

Рисунок 5.7.1 – Лист 1 «Решение ЗЛП двойственным методом»

По A_1 составляем двойственную задачу

$$y(y_1, y_2, y_3) = ?$$

$$Z(y) = 477y_1 + 441y_2 + 300y_3 \rightarrow \min$$

$$\begin{cases} 4y_1 + 1y_2 + 3y_3 \geq 21 \\ 6y_1 + 8y_2 + 2y_3 \geq 56 \\ y_i \geq 0 \end{cases}$$

$$y_1 = \frac{56}{13} \quad y_2 = \frac{49}{13} \quad y_3 = 0$$

$$Z_{\min} = 477 \cdot \frac{56}{13} + 441 \cdot \frac{49}{13} = 3717$$

$$F_{\max} = 21 \cdot 45 + 56 \cdot 49,5 = 3717$$

Вывод:

При производстве

$$x_1 = 21 \quad \text{и} \quad x_2 = 56$$

$$f_{\max} = 3717$$

Итого (4)

Рисунок 5.7.2 – Лист 2 «Решение ЗЛП двойственным методом»

Вывод

Работа в модуле линейного программирования позволила закрепить навыки применения математического аппарата для оптимизации технико-экономических процессов. В процессе выполнения заданий были построены экономико-математические модели производственной деятельности, а также успешно применены графический метод для визуального нахождения области допустимых решений ЗЛП с двумя переменными и универсальный симплексный метод для алгебраического расчета параметров. Полученный опыт подтвердил готовность к математическому обоснованию управленческих решений для эффективного распределения ресурсов.

ГЛАВА 6 ПРОЕКТИРОВАНИЕ И РАЗРАБОТКА ИНТЕРФЕЙСОВ

6.1 Индивидуальное задание. Верстка сайта «Мой не сам»

В данной работе мы верстали сайт для клининговой службы с использованием технологий HTML5 и CSS3 по макету.

С начала были добавлены все необходимые файлы и папки в проект. 5 файлов html (main.html, reg.html, auth.html, add_ap.html, forming_ap.html), два файла CSS: «normalize.css» (для обнуления стилей) и «style.css» (для создания своих стилей) и папка «images» для хранения картинок. Разработка навигации по сайту велась по готовой карте сайта (см. рис.6.1).

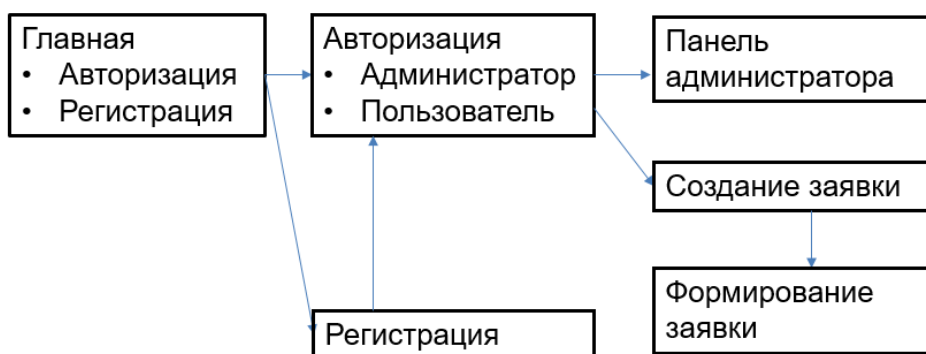


Рисунок 6.1 – «Путеводитель» по страницам сайта

В main.html были сверстаны картинка и две кнопки «Авторизация» и «Регистрация». На странице auth.html были сверстаны также две кнопки «Войти» и «Зарегистрироваться» и форма с двумя полями ввода информации «Логин» и «Пароль». На странице reg.html были сверстаны заголовок, две кнопки, а также форма с 5 полями для ввода:

- Логин.
- Пароль.
- ФИО.
- Телефон.

- Электронная почта.

Следующая страница `add_ap.html` включает в себя таблицу, поле для заполнения описания, селектор и кнопку «Создать заявку».

Далее страница `forming_ap.html` состоит из заголовка, кнопки «Оставить заявку» и формы с 6 полями для ввода:

- Адрес.
- Контактные данные.
- Дата.
- Время.
- Вид работ.
- Вид оплаты.

После заполнения всех полей пользователя переносит на страницу `waiting.html`. Она состоит из заголовка «Ожидайте прибытия мастера!» и зацикленной анимации фигур, сделанной в CSS.

Были созданы классы для двух видов кнопок (синяя и красная), но у этих классов есть анимация `hover`, при наведении на которую изменяется цвет кнопки и контур.

6.2 Валидация форм сайта «Мой не сам»

В данной работе были получены знания в области программирования на языке сценариев JavaScript.

Изначально JavaScript был создан, чтобы «сделать веб-страницы живыми». Программы на этом языке называются скриптами. Они могут встраиваться в HTML и выполняться автоматически при загрузке веб-страницы. Скрипты распространяются и выполняются, как простой текст. Им не нужна специальная подготовка или компиляция для запуска. Это отличает JavaScript от другого языка – Java. [8]

Непосредственно перед началом программирования, всем тегам `input` был выдан своё уникальное `id` имя. Далее была реализована проверка с помощью JavaScript на пустые поля для всех страниц. При нажатии на кнопку авторизоваться, зарегистрироваться или войти начиналась уникальная для каждой страницы проверка с помощью ветвлений (см. рис. 6.2). Если поля не заполнены, то перед пользователем появится окно с текстом о необходимости заполнить все поля.

```
12  function RegInputCheking (nextPage) {
13      if (document.getElementById('login').value=="" ||
14          document.getElementById('password').value=="" ||
15          document.getElementById('fio').value=="" ||
16          document.getElementById('tel').value=="" ||
17          document.getElementById('email').value=="")
18      {
19          alert("Заполните все поля!")
20          return
21      }
22      window.location.href = `${nextPage}`
23      return
24  }
```

Рисунок 6.2 – Проверка на заполнение в JavaScript

6.3 Flexbox-верстка сайта «Интерьер»

В данной работе была применена технология flex-box для создания гибких и адаптивных макетов веб-страниц. С начала были добавлены все необходимые файлы и папки в проект. Два файла CSS: «normalize.css» (для обнуления стилей) и «style.css» (для создания своих стилей), один файл html «index.html» и папка «images» для хранения картинок. Далее в html файле по порядку были созданы блоки «header», «main», «footer» и обернуты в div с классом «section».

Тэг `body` в который обернут `div` с классом «`section`» выравнивает все по центру, чтобы сам `div` находился по центру экрана и никуда не съезжал (см. рис.6.3).

```
5  body {  
6      display: flex;  
7      flex-direction: column;  
8      align-items: center;  
9  }
```

Рисунок 6.3 – CSS стиль тэга «`body`»

`Div` с классом «`section`» имел максимальную высоту и ограниченную максимальную ширину, также он аналогично тэгу «`body`» выравнивал все элементы внутри себя по центру (см. рис.6.4):

```
11  .container {  
12      height: 100%;  
13      max-width: 1200px;  
14      display: flex;  
15      flex-direction: column;  
16      align-items: center;  
17      gap: 100px;  
18  }
```

Рисунок 6.4 – CSS стиль `div`'а с классом «`container`»

После верстки всех блоков сайта, была начата верстка карточек. Основываясь на макете у карточки должно было быть:

1. картинка товара;
2. плашка с действиями (добавить в избранное, поделиться и т.п.);
3. название;
4. рейтинг;
5. цена (старая и новая).

И исходя из этих данных был сверстан следующий блок кода (см. рис.6.5).

```
49 <div class="card">
50   
51   <div class="other">
52     <a href="" class="other-a"><img src="" alt="img1"></a>
53     <a href="" class="other-a"><img src="" alt="img2"></a>
54     <a href="" class="other-a"><img src="" alt="img3"></a>
55   </div>
56   <div class="name-rating">
57     <p>Кресло</p>
58     <img src="" alt="rating img">
59   </div>
60   <div class="card-prices">
61     <p class="new-price">3500</p>
62   </div>
63 </div>
```

Рисунок 6.5 – HTML разметка карточки

После завершения основной работы был добавлен «интерактив» с использованием псевдокласса «hover» в CSS.

6.4 Слайдер HTML, CSS и JavaScript

В данной работе была применены технологии JavaScript и CSS для создания нескольких разных слайдеров. Практически все слайдеры сделаны по принципу проверки через ветвления. Для начала все картинки были переименованы по одному паттерну и имели в конце себя цифру например: «image_1.png». Это нужно для дальнейшего изменения пути к картинкам с помощью переменных.

Первый слайдер был реализован с помощью selector с тремя вариантами выбора и одного тега img. При выборе любого из трех вариантов в селекторе, происходит циклическая проверка и замена пути тега img к другой картинке в соответствии с номером нажатой кнопки.

Второй слайдер реализован с помощью тегов button с типом radio и img, а также CSS. При нажатии на любую кнопку путь до картинки меняется с помощью заготовленных CSS стилей и таким образом меняется картинка.

Третий слайдер реализован с помощью тегов button, img и JavaScript. На странице были размещены два тега button со стрелочками влево и вправо. В файле JavaScript были созданы две функции «ShowLeftImage()», «ShowRightImage()» и переменная j равная 1. При нажатии на кнопку «влево» переменная уменьшается на 1 и происходит проверка меньше либо равна переменной одному, если да, то переменная приравнивается трем (потому-что всего 3 картинки). Путь до картинки меняется с помощью кода JS, id тэга img «myImage» и переменной (см. рис. 6.6).

```
17  j= 1;
18  function ShowLeftImage() {
19      j--
20      if (j<1) j=3
21      myImage.src="images/image_"+ j +".png"
22  }
23
24  function ShowRightImage() {
25      j++
26      if (j>3) j=1
27      myImage.src="images/image_"+ j +".png"
28  }
```

Рисунок 6.6 – JavaScript код слайдера 3

Четвертый слайдер был реализован с помощью тегов img и script, а также JavaScript. На странице был размещен один тег img и два тега script. В первом теге script была создана переменная i равная 1 и функция «ShowNextImage()». Далее обращаясь по id тега img, изменялся путь до картинки с помощью переменной. Переменная циклически прибавляется и с помощью ветвления сбрасывается до первоначального значения, а вместе с переменной меняется картинка (см. рис. 6.7).


```

57 <div class="slider-4">
58   <script>
59     i=1
60     function ShowNextImage() {
61       gif_image.src="images/image_"+ i +".png"
62       i++
63       setTimeout ("ShowNextImage()", 250)
64       if (i>=4) i=1
65     }
66   </script>
67   
68   <script>ShowNextImage()</script>
69 </div>

```

Рисунок 6.7 – HTML верста и JavaScript код слайдера 4

6.5 Создание псевдотрехмерного пространства в CSS

Двумерное пространство – это геометрическая модель, характеризующаяся двумя измерениями – длиной и шириной, которые образуют плоскость. Как бы не было странным, двумерное пространство позволяет создать трехмерное пространство оставаясь двумерным. Таким пространством называется псевдотрехмерное пространство или же 2,5D. Этим приемом пользовались в 1990 годах, когда мощности компьютера не позволяли создавать 3D пространство например: компьютерная игра «Doom».

В ходе этого блока был создан сайт с кубами, повернутыми на 45 градусов относительно взгляда,двигающиеся вверх и вниз в разном порядке. Платформа с кубами состоит из блока perspective с основной платформой и 9 блоков cube, которые и являются кубами.

Основными CSS свойствами, отвечающими за формирование 3D блоков, являются transform и связанные с ним дополнительные параметры. Он позволяет изменять форму, размер, положение и поворот элементов не только в двумерном, но и трехмерном пространстве.

Следующим шагом идет создание анимации. Чтобы анимировать блоки необходимо использовать ранее изученное в ходе практики анимации при помощи keyframes. Для построения свойство в его заголовке необходимо

написать «@keyframes <название_переменной>» чтобы назначить анимации имя и пере использовать её далее. Далее следует каждому кубу задать свои правило применения анимации.

Чтобы не создавать каждому кубу отдельный класс можно применить модификатор «nth-child(номер_блока)» для выделения только одного блока. Внутри свойство каждого куба задаем свойство animation. Оно позволяет активировать анимацию и настроить ее проигрывание: «animation: <название_переменной_keyframes>, время анимации, задержка применения, конечное состояние, количество анимации». Чтобы, анимация была неравномерной стоит задать каждому кубу разную задержку применения и поставить бесконечное применение анимации.

6.6 Индивидуальное задание. Верстка сайта «Языковой портал»

Данная работа была нацелена на выявление фактических знаний. Верстка происходила на отключенном от интернета персональном компьютере с использованием технологий HTML5, CSS и JavaScript. Описание предметной области: сайт для школы, ведущей курсы иностранных языков.

С начала были добавлены все необходимые файлы и папки в проект. 3 файла html (main.html, form.html, courses.html), один файл CSS для создания своих стилей (style.css), один файл JavaScript, папка «style» для хранения CSS, папка «script» для хранения JavaScript файлов и приложение 1 «Прил_ДЭ_КОД 09.02.07-7-2026-M1». По техническому заданию разработка делилась на 2 модуля:

- 1) Проектирование и разработка информационных систем;
- 2) Разработка дизайна веб-приложений.

На основании описания предметной области были разработаны страницы основных пользовательских интерфейсов сайта с учетом требований к ним (Приложение А).

Т.к. работа рассчитана на 3 часа было принято решение делать все по порядку технического задания.

В рамках первого модуля длительностью 1 час 30 минут были сверстаны три основные веб-страницы:

1. Главная страница;
2. Страница «Курсы»;
3. Страница записи на бесплатный урок.

Главная страница. В шапке был размещен девиз школы и кнопка «Записаться на бесплатный урок». В центральном блоке отображен список ключевых преимуществ компании: информация о квалификации педагогов, график занятий в малых группах, а также отсутствие скрытых платежей. В подвал страницы были помещены контактные данные (адрес школы, контактный телефон и электронная почта). Также была создана навигационная ссылка для быстрого перехода на страницу «Курсы».

Страница «Курсы». В шапке была добавлена кнопка возврата со специальным текстом: «Еще не выбрал? Вернуться». На ней был сверстан блок с карточками курсов. Каждая карточка содержит: наименование, графическое изображение, описание и стоимость обучения. В подвале страницы продублирована контактная информация.

Страница записи на бесплатный урок. Данная страница содержит форму с полями:

1. ФИО;
2. Номер телефона;
3. Электронная почта;
4. Выбор курса;
5. Согласие на обработку персональных данных;
6. Кнопка для отправки данных.

С помощью HTML-тегов `input` и `select` пользователю предоставлена возможность указать ФИО, выбрать желаемый курс и поставить обязательную галочку `checkbox` согласия на обработку персональных данных. Кнопка

отправки формы «Записаться» настроена на автоматический перенаправление обратно на главную страницу.

По завершении первого модуля все созданные файлы HTML кода были заархивированы и переименованы согласно регламенту: Лобанов_П-28_Модуль1.zip.

Второй модуль задания рассчитан на 1 час 30 минут. Он был посвящен визуальной модернизации, оптимизации графики, добавлению калькулятора стоимости и адаптации страниц под мобильное разрешение смартфона (390×844 пикселей).

Для реализации адаптивности в файле style.css были применены медиа-запросы @media (max-width: 390px). Все изображения из архива «Прил_ДЭ_КОД 09.02.07-7-2026-M1» были переименованы для более понятного и читаемого кода.

На главной странице был реализован автоматический слайдер из четырех картинок одинакового размера. С помощью функции JavaScript setInterval была настроена циклическая смена кадров каждые 3 секунды. Дополнительно слева и справа от слайдера были созданы стрелки-управления «Вперед» и «Назад» с помощью элементов Button для ручного перелистывания.

На Странице «Курсы» под витриной программ был спроектирован и запрограммирован интерактивный JS-калькулятор примерной стоимости обучения. Скрипт считывает динамические изменения из трех полей выбора:

- Продолжительность одного занятия;
- Общее количество занятий;
- Выбранный тариф «выгодный», «комфортный» или «усиленный».

Для привлечения внимания к кнопкам применили CSS-анимацию плавного масштабирования и изменения цвета.

На странице записи для всех полей ввода была настроена проверка на их заполненность с помощью JavaScript.

Итоговые файлы второго модуля были заархивированы в файл Лобанов_П-28_Модуль2.zip.

Вывод

Данная глава посвящена освоению современных веб-технологий клиентской разработки (Frontend) и прохождению комплексного индивидуального задания. В ходе работы были детально отработаны навыки семантической верстки HTML/CSS, технологии Flexbox, создания CSS-анимаций и псевдотрехмерного пространства. Использование языка JavaScript позволило реализовать динамическую валидацию форм и логику интерактивных элементов (слайдеров и онлайн-калькулятора). Успешная адаптация проекта «Языковой портал» под мобильные разрешения в условиях ограниченного времени подтвердила квалификацию в области создания удобных и гибких веб-интерфейсов.

ЗАКЛЮЧЕНИЕ

В ходе прохождения учебной практики в ЧОУВО «Международный Институт Дизайна и Сервиса» были полностью выполнены поставленные задачи и достигнута цель освоения видов профессиональной деятельности по специальности 09.02.07 «Информационные системы и программирование».

За время практики были закреплены и усовершенствованы следующие практические навыки и профессиональные компетенции:

- В области настольной разработки: освоен фреймворк Windows Forms на языке C#, проведен аудит готового ПО, реализованы системы валидации данных, фильтрации и сортировки в рамках создания приложений «Учебный контроль» и «РиелторПро».
- В области мобильной разработки: изучена архитектура ОС Android и жизненный цикл Activity; освоен декларативный подход к проектированию интерфейсов на языке Kotlin с использованием Jetpack Compose, ленивых списков и динамической навигации.
- В области реляционных баз данных: пройдены этапы инфологического и логического проектирования систем складского учета, определены сущности, связи и ограничения целостности данных для последующей физической реализации в СУБД.
- В области математического моделирования: освоены методы линейного программирования для оптимизации производственных процессов, включая построение экономико-математических моделей, графический и симплексный методы решения задач линейного программирования.
- В области веб-разработки: отработаны навыки семантической верстки HTML5/CSS3 с применением технологии Flexbox и псевдотрехмерной анимации; на языке JavaScript реализована клиентская логика валидации форм, интерактивных слайдеров и онлайн-калькулятора стоимости для проекта «Языковой портал».

Приобретенный практический опыт и сформированные общие и профессиональные компетенции соответствуют квалификационным требованиям, предъявляемым к программисту, и подтверждают готовность к самостоятельной профессиональной деятельности в сфере разработки программного обеспечения и веб-приложений.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Бурзуева Н. Н. Мостовой Я. А. Анализ надежности среды разработки Android Studio // CyberLeninka [Электронный ресурс]. – Режим доступа: <https://cyberleninka.ru/article/n/analiz-nadezhnosti-sredy-razrabotki-android-studio>
2. Градусов, А. Б. БАЗЫ ДАННЫХ. Проектирование баз данных [Электронный ресурс] : учеб.-практ. пособие / А. Б. Градусов, А. В. Шутов ; Владим. гос. ун-т им. А. Г. и Н. Г. Столетовых. – Владимир : Изд-во ВлГУ, 2024. – 259 с.
3. Документация Android. Жизненный цикл деятельности Android – [Электронный ресурс]. – Режим доступа: <https://developer.android.com/guide/components/activities/activity-lifecycle?hl=ru>
4. Документация Kotlin. Обзор базового синтаксиса Kotlin – [Электронный ресурс]. – Режим доступа: <https://kotlinlang.org/docs/basic-syntax.html#variables>
5. Лысенко К.Д. Инфологическое и даталогическое проектирование информационной системы спортивной школы с помощью erwin // CyberLeninka [Электронный ресурс]. – Режим доступа: <https://cyberleninka.ru/article/n/infologicheskoe-i-datalogicheskoe-proektirovanie-informatsionnoy-sistemy-sportivnoy-shkoly-s-pomoschyu-erwin>
6. Мирошниченко Е. А. К формальному определению понятия база данных // CyberLeninka [Электронный ресурс]. – Режим доступа: <https://cyberleninka.ru/article/n/k-formalnomu-opredeleniyu-ponyatiya-baza-dannyh>
7. Рахматуллин Т.Г. Kotlin как язык программирования будущего // CyberLeninka [Электронный ресурс]. – Режим доступа: <https://cyberleninka.ru/article/n/kotlin-kak-yazyk-programmirovaniya-buduschego>
8. Современный учебник JavaScript. Введение в JavaScript [Электронный ресурс]. – Режим доступа: <https://learn.javascript.ru/intro>.

9. Microsoft Docs. Документация по Windows Forms – [Электронный ресурс]. – Режим доступа: <https://learn.microsoft.com/ru-ru/dotnet/desktop/winforms/>

10. Microsoft Docs. Язык программирования C# – [Электронный ресурс]. – Режим доступа: <https://learn.microsoft.com/ru-ru/dotnet/csharp/>

ПРИЛОЖЕНИЕ А

Описание предметной области:

Сайт для школы, ведущей курсы иностранных языков «Языковой портал».

Главная страница содержит девиз школы и кнопку «Записаться на бесплатный урок», преимущества школы, слайдер с фото и ссылку для перехода на страницу «Курсы».

Преимущества компании:

- Курсы ведут квалифицированные педагоги, многие из которых имеют опыт работы за рубежом или являются носителями языка.
- Возможность выбора удобного графика занятий в группах с комфортной наполняемостью (6-8 человек) или заниматься индивидуально.
- Отсутствие скрытых платежей, гибкая система скидок.

Школа «Языковой портал» предоставляет возможность пройти обучение на платных курсах изучения иностранных языков. Клиенты выбирают курс и записываются на него, указав следующие данные: ФИО, телефон, адрес электронной почты, выбор курса. После чего менеджер школы уточняет детали, созвонившись с клиентом. После этого клиент производит оплату, предоставив платежный документ об оплате менеджеру.

Менеджер записывает его на курс. После окончания курсов слушатель получает соответствующий сертификат. Менеджер ведет учет слушателей, списков групп, а также делает рассылку слушателям по электронной почте при переносе или отмене занятий.

Модуль № 1 Проектирование и разработка информационных систем.

Продолжительность выполнения модуля задания: 1 ч. 30 мин.

На основании описания предметной области разработайте страницы основных пользовательских интерфейсов приложения.

Требования к интерфейсам приложения:

Главная страница девиз школы и кнопка «Записаться на бесплатный урок», преимущества компании, контактная информация (адрес, телефон, адрес электронной почты) находится в подвале сайта, переход на страницу «Курсы».

Страница «Курсы».

На странице расположена информация об актуальных курсах (наименование, примерная стоимость, краткая характеристика, изображение).

Контактная информация находится в подвале (адрес, телефон, адрес электронной почты). Предусмотрите навигацию между главной страницей и страницей «Курсы» в виде кнопки или ссылки с текстом «Еще не выбрал? Вернуться».

Страница записи на бесплатный урок.

Клиенту необходимо предоставить возможность ввести ФИО, телефон (в формате +7(XXX)-XXX-XX-XX) и адрес электронной почты (формат электронной почты), выбор курса, согласие на обработку персональных данных. Кнопка «Записаться» возвращает на главную страницу.

1. Сохранить работу в архив с именем Фамилия_группа_Модуль1.zip
 2. Необходимые приложения: Прил_ДЭ_КОД 09.02.07-7-2026-M1.zip
- Модуль № 2 Разработка дизайна веб-приложений. Продолжительность выполнения модуля задания: 1 ч. 30 мин.

Вашей задачей является усовершенствование внешнего вида существующего приложения. Вам предстоит провести работу по оптимизации изображений, разместить значки, формы полей, кнопки, ссылки и прочие графические элементы таким образом, чтобы интерфейс стал привлекательным и удобным для пользователя.

Используйте современные инструменты и библиотеки, чтобы повысить качество графического оформления приложения. Обеспечьте совместимость вашего дизайна с экраном смартфона размером 390×844 пикселей. Готовые

макеты можно оформить в виде отдельных HTML-файлов, каждый из которых соответствует отдельной странице сайта.

Главная страница

Заказчик также желает видеть в приложении слайдер, который будет автоматически сменять изображения с интервалом в 3 секунды. Слайдер должен иметь одинаковые по размеру 4 изображения, элементы управления (вперед/назад) и быть органично вписан в дизайн приложения.

Страница «Курсы».

На странице расположена информация об актуальных курсах (наименование, примерная стоимость, краткая характеристика, изображение) и калькулятор расчета примерной стоимости курса (выбор продолжительности занятия, количества занятий и тарифа (выгодный, комфортный, усиленный).

Инструкция к выполнению практической части:

Используйте анимацию для привлечения внимания посетителя к акцентам и основным объектам приложения.

В данном модуле уделите внимание дизайну. Вам предоставлены изображения, которые необходимо оптимизировать и улучшить, чтобы достичь основную цель - создание идеальной информационной системы. Разместите изображения, иконки, поля форм, кнопки, ссылки, графические элементы так, чтобы они дополняли приложение.

Вам необходимо также разработать дизайн всех страниц для использования со смартфоном с разрешением 390x844 px. Дизайн представить в виде .html файлов (отдельный файл для каждой страницы).

1. Сохранить работу в архив с именем Фамилия_группа_Модуль2.zip
2. Необходимые приложения: Прил_ДЭ_КОД 09.02.07-7-2026-M1.zip